

CS 284: Homework 2

Due: 2026-03-09

This is a paired assignment with Programming Assignment 2. It is recommended that you read through both assignments before beginning, and that you work on them in parallel.

In Programming Assignment 2, you are implementing a `TriangularList`. But just implementing it is not enough. We also wish to implement it efficiently, and to know how efficient it is.

In this homework, you will analyse the data structure you have implemented, as well as potentially improve your implementation.

For this Homework, and all following ones, you must justify your work (unless stated otherwise). Giving the answer without an explanation for why your answer is correct will not receive full credit.

Problem 1: To begin, let us try to calculate how quickly a `TriangularList` grows as more elements are added. (For this entire Homework and Programming Assignment, you may find the following mathematical identity useful: $1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$.)

- Given a `TriangularList` with 102 elements, how long does the `metaList` have to be? In other words, how many inner lists are necessary to store 102 total elements? (You do not have to justify your answer.)
- Given a `metaList` with k inner lists, what is the maximum number of total elements that can be stored?
- Given a `TriangularList` with n elements, how long does the `metaList` have to be. Your answer can be in Big-O notation.

Problem 2: Next, let us analyse the runtime of your `TriangularList`. For each of the following methods, give the runtime complexity of your implementation on each of these. Remember to justify your work, and to take into account the runtime of the underlying `Lists` used.

- The `get` method.
- The `insert` method.
- The `remove` method.
- The `size` method.

Problem 3: As mentioned before, your `metaList` could be any combination of two `List` types. It could be a `LinkedList` of `ArrayLists`, an `ArrayList` of `LinkedLists`, or any other.

However, these different implementations notably do not perform the same way. In this task, your goal is to compare your configuration with the other possible ones.

- Which implementation did you choose to go with?
- Why did you decide to go with the above implementation?
- What is the runtime complexity of `insert` for the four possible configurations?